

Assignment - 5

1. Write a program that asks the user to enter some text and saves it to a file called output.txt. Then, open the file and read its contents, printing them to the console.

Code:

```
import os

# Read and Write functions

def write_txt(filename):
    text = input("Enter some text to save into the file: ")
    with open(filename, "w" ) as file:
        file.write(text)
        print("Text is successfully written into the file")

def read_text(filename):
    with open(filename, "r") as file:
        content = file.read()
        print(content)

# Driver code
filename = "output.txt"
write_txt(filename)
print("The content of the file are: ")
read_text(filename)
```

Output:

Output.txt is created in the same directory. And the same text is saved in the file.

"C:\Wipro Training\Python\PythonBasics\venv\Scripts\python.exe" "C:\Wipro Training\Python\PythonBasics\Assignments\day_5\txt_file.py"

Enter some text to save into the file: Hello Wipro!! This is Deekshita.

Text is successfully written into the file

The content of the file are:

Hello Wipro!! This is Deekshita.

2. Write a program that reads a text file called sample.txt and counts the number of lines, words, and characters in the file. Print the counts.

Created a sample.txt with some text inside it so that program can read.

Code:

```
import os

# Reading the file

with open("sample.txt", 'r') as file:
    lines = file.readlines()

    count_lines = len(lines)
    count_words = sum(len(line.split()) for line in lines)
    count_char = sum(len(line) for line in lines)

print(f"Number lines: {count_lines}")
print(f"Number of words: {count_words}")
print(f"Number of characters: {count_char}")
```

sample.txt file

```
The sun rises from the east.
Japan sees the first sun rise.
The sun never sets in the Britain Empire.
```

Output:

```
"C:\Wipro Training\Python\PythonBasics\venv\Scripts\python.exe" "C:\Wipro
Training\Python\PythonBasics\Assignments\day_5\count_lwc.py"
Number lines: 3
Number of words: 20
Number of characters: 101
```

3. Write a program that reads the contents of a file called source.txt and writes the contents to another file called destination.txt. Ensure that destination.txt is created if it doesn't exist.

Code:

```
import os

with open("sources.txt", "r") as file:
    content = file.read()
    print("The content is read successfully")
```

```
with open("destination.txt", "w") as file:
    file.write(content) # If the file does not exist then it will create the
file
    print("The content is successfully copied into this file")
```

sources.txt

```
Hello Trainees!!
Welcome to the Milestone one
```

destination.txt after coping

```
Hello Trainees!!
Welcome to the Milestone one
```

Output:

```
"C:\Wipro Training\Python\PythonBasics\venv\Scripts\python.exe" "C:\Wipro
Training\Python\PythonBasics\Assignments\day_5\file_transfer.py"
The content is read successfully
The content is successfully copied into this file
```

4. Write a program that appends a line of text to a file called log.txt. After appending the text, open the file and print its contents to verify that the text was added.

Code:

```
import os

# Functions to append and read

def append_text(filename, content):
    with open(filename, "a") as file:
        file.write(content)
        print("The content is successfully appended")

def read_text(filename):
    with open(filename, "r") as file:
        context = file.read()
        print(context)

# Driver code
if __name__ == "__main__":
    filename = "log.txt"

    content = '\n\nThe order for 5 Laptops and 3 Desktops is received'
    append_text(filename, content)
    print(f"After appending the file content is:")
    read_text(filename)
```

log.txt before appending

```
Order for 5 Laptops is placed  
Order for 3 Desktops is placed
```

log.txt after appending

```
Order for 5 Laptops is placed  
Order for 3 Desktops is placed  
The order for 5 Laptops and 3 Desktops is received
```

Output:

```
"C:\Wipro Training\Python\PythonBasics\.venv\Scripts\python.exe" "C:\Wipro  
Training\Python\PythonBasics\Assignments\day_5\file_append.py"  
The content is successfully appended  
After appending the file content is:  
Order for 5 Laptops is placed  
Order for 3 Desktops is placed  
The order for 5 Laptops and 3 Desktops is received
```

5. Write a program that asks the user to input a number in the form of a string. Use a try - except block to convert the string to an integer. If a ValueError occurs (e.g., if the user inputs a non-numeric string), print an error message. Otherwise, print the integer.

Code:

```
# Try and Except  
  
user_string = input("Enter a string: ")  
  
try:  
    number = int(user_string)  
    print(f"Successfully converted string to number. Your number is: {number}")  
except ValueError:  
    print("Error!! The string does not complete numeric values")
```

Outputs:

```
Enter a string: Hello World  
Error!! The string does not complete numeric values
```

```
Enter a string: 1264537  
Successfully converted string to number. Your number is: 1264537
```

6. Write a program that tries to open a file specified by the user for reading. Use a try except block to handle FileNotFoundError if the file does not exist. If the file is successfully opened, print its contents; otherwise, print an error message.

Code:

```
import os

filename = input("Enter a filename: ")

try:
    with open(filename, "r") as file:
        content = file.read()
        print(f"The content of the file are: {content}")
except FileNotFoundError:
    print(f"Error!! The {filename} file is NOT FOUND ")
```

Outputs:

```
"C:\Wipro Training\Python\PythonBasics\venv\Scripts\python.exe" "C:\Wipro
Training\Python\PythonBasics\Assignments\day_5\try_exp_file.py"
Enter a filename: dimple.txt
Error!! The dimple.txt file is NOT FOUND
```

```
"C:\Wipro Training\Python\PythonBasics\venv\Scripts\python.exe" "C:\Wipro
Training\Python\PythonBasics\Assignments\day_5\try_exp_file.py"
Enter a filename: sample.txt
The content of the file are: The sun rises from the east.
Japan sees the first sun rise.
The sun never sets in the Britain Empire.
```

7. Write a program that asks the user to input two numbers and performs division. Use a try-except block to handle both ZeroDivisionError and ValueError. Print different messages for each exception. If no exception occurs, print the result of the division.

Code:

```
# ZeroDivisionError and ValueError
try:
    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))
```

```

div = num1/num2
print(f"The division of {num1} and {num2} is: {div}")

except ZeroDivisionError:
    print("Error!! Cannot divide number with zero")

except ValueError:
    print("Error!! Please enter a valid number(non-string)")

```

Outputs:

"C:\Wipro Training\Python\PythonBasics\venv\Scripts\python.exe" "C:\Wipro Training\Python\PythonBasics\Assignments\day_5\try_exp_nums.py"

Enter first number: 25

Enter second number: 5

The division of 25 and 5 is: 5.0

"C:\Wipro Training\Python\PythonBasics\venv\Scripts\python.exe" "C:\Wipro Training\Python\PythonBasics\Assignments\day_5\try_exp_nums.py"

Enter first number: 5

Enter second number: 0

Error!! Cannot divide number with zero

"C:\Wipro Training\Python\PythonBasics\venv\Scripts\python.exe" "C:\Wipro Training\Python\PythonBasics\Assignments\day_5\try_exp_nums.py"

Enter first number: 7

Enter second number: t

Error!! Please enter a valid number(non-string)

8. Write a program that takes a list of numbers and asks the user to input an index to access an element from the list. Use a try-except block to handle IndexError if the user enters an invalid index. Print the corresponding element if the index is valid; otherwise, print an error message.

Code:

```

# Try and except with Lists

try:
    lst = list(map(int,input("Enter numbers with spaces: ").split()))
    idx = int(input("Enter a index value number: "))

    print(f"The element at index {idx} is: {lst[idx]}")

```

```
except IndexError:
    print("Error!! Enter the correct index value")
```

Outputs:

"C:\Wipro Training\Python\PythonBasics\.venv\Scripts\python.exe" "C:\Wipro Training\Python\PythonBasics\Assignments\day_5\try_exp_list.py"
 Enter numbers with spaces: 5 7 3 4 8 9
 Enter a index value number: 7
 Error!! Enter the correct index value

"C:\Wipro Training\Python\PythonBasics\.venv\Scripts\python.exe" "C:\Wipro Training\Python\PythonBasics\Assignments\day_5\try_exp_list.py"
 Enter numbers with spaces: 3 4 6 2 8 6 9
 Enter a index value number: 4
 The element at index 4 is: 8

9. Write a program that defines a custom exception class `NegativeNumberError`. The program should ask the user to input a positive number. If the user enters a negative number, raise the `NegativeNumberError` and handle it using a try-except block, printing an appropriate error message.

Code:

```
class NegativeNumberError(Exception):
    pass

try:
    num = int(input("Enter a positive number: "))
    if num < 0:
        raise NegativeNumberError("Negative numbers are not allowed!!")
    print(f"You entered: {num}")

except NegativeNumberError as e:
    print(f"Custom Error: {e}")

except ValueError:
    print("Error!! Please enter a valid number.")
```

Outputs:

"C:\Wipro Training\Python\PythonBasics\.venv\Scripts\python.exe" "C:\Wipro Training\Python\PythonBasics\Assignments\day_5\custom_exp.py"
 Enter a positive number: 15
 You entered: 15

"C:\Wipro Training\Python\PythonBasics\venv\Scripts\python.exe" "C:\Wipro Training\Python\PythonBasics\Assignments\day_5\custom_exp.py"
Enter a positive number: -5
Custom Error: Negative numbers are not allowed!!

10. Write a program that repeatedly asks the user to input two numbers and performs division. Use a try-except block inside a loop to handle ZeroDivisionError and ValueError. The program should continue until the user provides valid input and a valid division result is printed.

Code:

```
while True:
    try:
        num1 = int(input("Enter first number: "))
        num2 = int(input("Enter second number: "))
        res = num1/num2

        print(f"Division result is: {res}")
        break

    except ValueError:
        print("Invalid input!! Please enter number only.")

    except ZeroDivisionError:
        print("Error! Cannot divide with zero. Try again.")
```

Outputs:

"C:\Wipro Training\Python\PythonBasics\venv\Scripts\python.exe" "C:\Wipro Training\Python\PythonBasics\Assignments\day_5\try_exp_loop.py"
Enter first number: 34
Enter second number: 2
Division result is: 17.0

"C:\Wipro Training\Python\PythonBasics\venv\Scripts\python.exe" "C:\Wipro Training\Python\PythonBasics\Assignments\day_5\try_exp_loop.py"
Enter first number: 4
Enter second number: 0
Error! Cannot divide with zero. Try again.
Enter first number: 4
Enter second number: t
Invalid input!! Please enter number only.
Enter first number: 4
Enter second number: 2
Division result is: 2.0

11. Write a program that tries to open and read a file. Use a try-except-finally block to handle potential exceptions like FileNotFoundError. Ensure that the finally block prints a message indicating that the program has completed, whether an exception occurred or not.

Code:

```
from logging import exception

filename = input("Enter the filename to read: ")

try:
    file = open(filename, "r")
    print(file.read())
    file.close()

except FileNotFoundError:
    print(f"ERROR!! {filename} was not found.")

except Exception as e:
    print(f"An unexpected error occurred: {e}")

finally:
    print("The Execution completed: File operation attempt finished.")
```

Outputs:

```
"C:\Wipro Training\Python\PythonBasics\.venv\Scripts\python.exe" "C:\Wipro
Training\Python\PythonBasics\Assignments\day_5\t_e_finally.py"
Enter the filename to read: dimple.txt
ERROR!! dimple.txt was not found.
The Execution completed: File operation attempt finished.
```

```
"C:\Wipro Training\Python\PythonBasics\.venv\Scripts\python.exe" "C:\Wipro
Training\Python\PythonBasics\Assignments\day_5\t_e_finally.py"
Enter the filename to read: sample.txt
The sun rises from the east.
Japan sees the first sun rise.
The sun never sets in the Britain Empire.
The Execution completed: File operation attempt finished.
```

12. Write a program that defines a function to add two numbers. Use a try-except block to handle TypeError in case the function is called with non-numeric arguments (e.g., strings). Print an appropriate error message if the exception is caught.

Code:

```
def add_numbers(a, b):  
    try:  
        result = a + b  
        print(f"The sum of {a} and {b} is: {result}")  
  
    except TypeError:  
        print("ERROR!! Both arguments must be numbers.")  
  
# Driver code  
  
if __name__ == "__main__":  
    print("Test 1 (Valid): ")  
    add_numbers(10,15)  
  
    print("Test 2 (Invalid): ")  
    add_numbers(9, 'five')
```

Output:

"C:\Wipro Training\Python\PythonBasics\venv\Scripts\python.exe" "C:\Wipro Training\Python\PythonBasics\Assignments\day_5\add_try_exp.py"
Test 1 (Valid):
The sum of 10 and 15 is: 25
Test 2 (Invalid):
ERROR!! Both arguments must be numbers.

13. Write a program that repeatedly asks the user to input an integer. Use a `try-except` block to handle `ValueError` in case the user enters a non-integer value. The program should keep asking for input until a valid integer is provided, and then print the integer.

Code:

```
while True:  
    try:  
        number = int(input("Enter a number: "))  
        print(f"Your number is: {number}")  
        break  
  
    except ValueError:  
        print("ERROR!! Invalid input. Please enter numeric value only. Try again.")
```

Output:

```
"C:\Wipro Training\Python\PythonBasics\.venv\Scripts\python.exe" "C:\Wipro Training\Python\PythonBasics\Assignments\day_5\try_exp_loop2.py"
```

```
Enter a number: five
```

```
ERROR!! Invalid input. Please enter numeric value only. Try again.
```

```
Enter a number: r4t5
```

```
ERROR!! Invalid input. Please enter numeric value only. Try again.
```

```
Enter a number: 25\
```

```
ERROR!! Invalid input. Please enter numeric value only. Try again.
```

```
Enter a number: 25
```

```
Your number is: 25
```